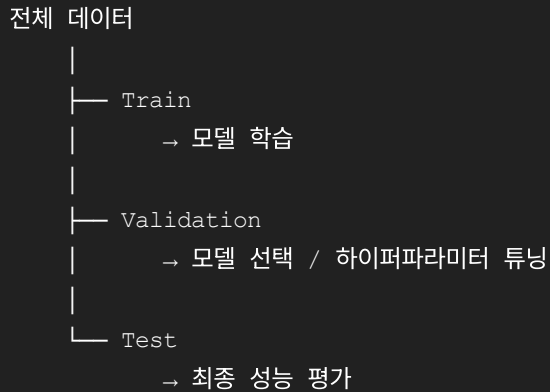


좋습니다. 지금까지의 대화를 기준으로 학생 강의용 PPT 흐름으로 정리하면 다음과 같이 구성하는 것이 가장 이해하기 쉽습니다.

PPT 구성안: Cross Validation과 Validation의 역할

슬라이드 1. 머신러닝 데이터 분할의 목적



핵심 질문

왜 Train과 Test만 사용하지 않고 Validation이 필요한가?

슬라이드 2. Train / Validation / Test의 역할

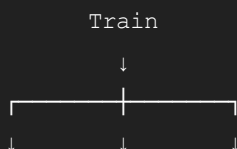
데이터	역할	모델이 보는가?
Train	모델 파라미터 학습	O
Validation	모델/하이퍼파라미터 선택	O
Test	최종 일반화 성능 평가	X

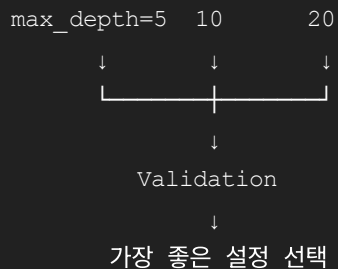
핵심

Validation은 모델을 선택하기 위한 데이터이다.

슬라이드 3. Validation이 필요한 이유

예를 들어 Random Forest의 하이퍼파라미터를 결정해야 한다.





즉,

Train 데이터만으로는 어떤 설정이 새로운 데이터에서도 좋은지 판단하기 어렵다.

슬라이드 4. 하지만 Validation Set에는 문제가 있다

예를 들어:

전체 데이터 1,000개

Train	800
Validation	100
Test	100

Validation을 별도로 떼어 놓으면

- 실제 모델 학습에 사용할 데이터가 감소
- 어떤 데이터가 Validation에 들어갔느냐에 따라 성능이 달라질 수 있음
- 데이터가 적을수록 비효율적

따라서 등장한 방법이

K-Fold Cross Validation

슬라이드 5. K-Fold Cross Validation

Train 데이터를 K개로 나눈다.

Train Data

Fold 1 | Fold 2 | Fold 3 | Fold 4 | Fold 5

5-Fold라면 매번 하나를 Validation으로 사용한다.

1회: [Validation] [Train] [Train] [Train] [Train]

```
2회: [Train] [Validation] [Train] [Train] [Train]

3회: [Train] [Train] [Validation] [Train] [Train]

4회: [Train] [Train] [Train] [Validation] [Train]

5회: [Train] [Train] [Train] [Train] [Validation]
```

모든 데이터가 한 번씩 **Validation** 역할을 수행한다.

슬라이드 6. K-Fold의 핵심 목적

각 Fold에서 모델을 학습하고 성능을 측정한다.

```
Fold 1 → 0.91
Fold 2 → 0.89
Fold 3 → 0.93
Fold 4 → 0.90
Fold 5 → 0.92
      ↓
평균 = 0.91
```

따라서

K개의 Validation 성능 평균을 이용하여 모델/하이퍼파라미터를 평가한다.

슬라이드 7. 중요한 오해: K-Fold에서 최종 모델을 고르는가?

아니다.

K-Fold에서는 각각 다른 데이터로 학습하기 때문에

```
Fold 1 → Model 1
Fold 2 → Model 2
Fold 3 → Model 3
Fold 4 → Model 4
Fold 5 → Model 5
```

각 모델은 서로 다른 학습 데이터를 사용한다.

따라서

5개 모델 중 하나를 최종 모델로 선택하는 것이 아니다.

슬라이드 8. K-Fold에서 실제로 선택하는 것

K-Fold가 선택하는 것은 하이퍼파라미터이다.

예:

```
Random Forest
```

```
max_depth = 5    → CV 평균 0.87  
max_depth = 10   → CV 평균 0.91 ← 선택  
max_depth = 20   → CV 평균 0.89
```

따라서:

```
최적 하이퍼파라미터  
max_depth = 10
```

을 결정한다.

핵심

K-Fold는 "어떤 모델 설정이 좋은가?"를 결정하는 과정이다.

슬라이드 9. 그렇다면 Refit이란 무엇인가?

최적 하이퍼파라미터를 결정한 후

```
max_depth = 10  
n_estimators = 200
```

이 설정으로 새로운 Random Forest를 만든다.

그리고 이번에는:

Fold 데이터 80% ❌

전체 Train 데이터 100% ○

를 사용하여 다시 학습한다.

이것이 Refit이다.

슬라이드 10. "이미 모델이 있는데 왜 다시 학습하는가?"

이 질문이 핵심이다.

K-Fold에서 만들어진 모델:

```
Model 1 → Train 80%
Model 2 → Train 80%
Model 3 → Train 80%
Model 4 → Train 80%
Model 5 → Train 80%
```

이 모델들은 검증을 위한 임시 모델이다.

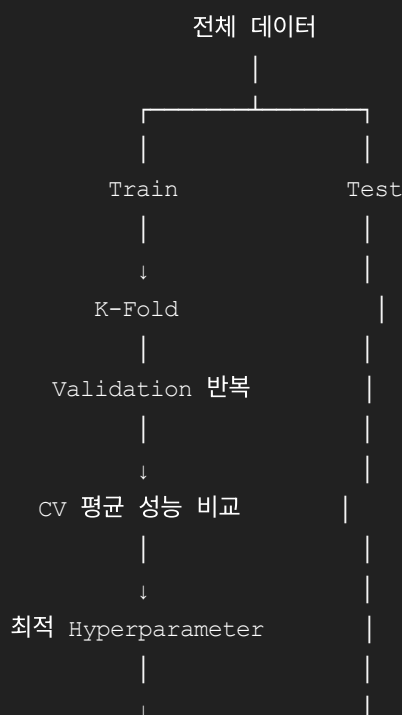
최적 설정을 찾은 후:

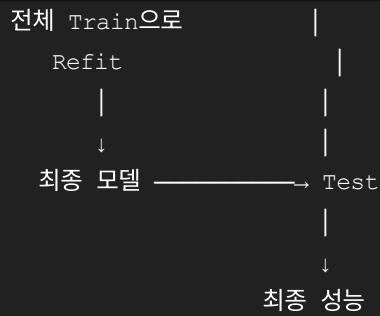
```
최적 설정
  ↓
새로운 모델 생성
  ↓
전체 Train 100%로 학습
  ↓
최종 모델
```

즉,

Refit은 기존 모델을 이어서 학습하는 것이 아니라, 같은 알고리즘 + 최적 하이퍼파라미터로 새로운 모델을 전체 Train 데이터에서 다시 학습하는 것이다.

슬라이드 11. 전체 머신러닝 과정





슬라이드 12. K-Fold를 사용하면 Validation Set은 필요 없는가?

별도의 Validation Set은 일반적으로 필요 없다.

왜냐하면:

```

Train 데이터
  ↓
  K-Fold
  ↓
  각 Fold가 Validation 역할
  
```

즉,

Validation이라는 역할이 사라지는 것이 아니라 K-Fold 내부에서 수행되는 것이다.

슬라이드 13. 일반적인 Validation vs K-Fold

구분	Hold-out Validation	K-Fold CV
Train	일부 사용	K-1개 Fold
Validation	한 번	K번 반복
데이터 활용	상대적으로 낮음	높음
계산량	적음	많음
작은 데이터	불리할 수 있음	유리
하이퍼파라미터 선택	가능	가능

슬라이드 14. Test 데이터는 왜 별도로 필요한가?

Test는 최종 시험지이다.

```
K-Fold
  ↓
하이퍼파라미터 결정
  ↓
전체 Train Refit
  ↓
최종 모델
  ↓
Test
  ↓
최종 성능
```

Test를 보면서 계속 모델을 수정하면:

```
Test → 평가
Test → 수정
Test → 평가
Test → 수정
```

Test가 사실상 Validation이 되어버린다.

따라서

Test는 최종 단계에서만 사용한다.

슬라이드 15. 최종 정리 ★

```
TRAIN
  ↓
모델이 학습한다
  ↓
K-FOLD
  ↓
Validation을 반복 수행한다
  ↓
CV 평균 성능으로
최적 Hyperparameter를 선택한다
  ↓
REFIT
  ↓
전체 Train 데이터로
새로운 최종 모델을 학습한다
  ↓
TEST
```

↓
최종 일반화 성능을 평가한다

학생들에게 기억시킬 4문장

Train = 공부

Validation = 어떤 설정이 좋은지 확인

K-Fold = **Validation**을 여러 번 바꿔가며 확인

Test = 마지막 시험

그리고 가장 중요한 한 문장:

K-Fold에서 선택하는 것은 '이미 학습된 모델'이 아니라 '최적 하이퍼파라미터'이며, 그 설정으로 전체 **Train** 데이터를 이용해 최종 모델을 다시 학습한다.

사용량이 PM 5:40에 재설정될 때까지 파일, 이미지, 데이터 분석을 사용할 수 없습니다. 텍스트로만 계속 채팅하거나 더 많이 이용하려면 업그레이드하세요.

[Plus 다시 가입](#)



HP Korea

광고

로컬AI 개발 AI Station

합리적인 비용으로 온디바이스 AI 개발 환경을 구축하세요.